

---

## Correction de la série de TD ASD N°4 (Les tableaux et les sous-programmes)

---

### EXERCICE 1

---

#### Algorithme Ex1

**Constante** max=100

**Variables**

i, n1, u, x2, nb1, occ1 : entier  
 trouve : booléen  
 T : Tableau [1.. max] de entier

**Début**

**Répéter**

Ecrire("donner un entier positif")  
 Lire(n1)

**Jusqu'à** (n1>0 et n1<=max)

nb1 ← 0

occ1 ← 0

**pour** i **de** 1 **à** nb1 **faire**

x2 ← i

trouve ← faux

**répéter**

u ← x2 mod 10

x2 ← x2 div 10

**si** (u=1) **alors**

occ1 ← occ1 + 1

trouve ← vrai

**finSi**

**jusqu'à** (x2=0)

**si** (trouve) **alors**

nb1 ← nb1+1

T[nb1] ← i

**FinSi**

**Finpour**

Ecrire(" Nbre de chiffres 1 : ",occ1)

Ecrire(" N ",nb1)

**Pour** i **de** 1 **à** nb1 **faire**

Ecrire(" T[" ,i, "]: ",T[i])

**FinPour**

**Fin**

### Autre Solution Avec les sous Programmes

**Fonction** Vérif(n :entier) : entier

**Variable** u, occ: entier

**Début**

occ ← 0

**répéter**

u ← n mod 10

**si** (u=1) **alors**

occ ← occ + 1

**finSi**

n ← n div 10

**jusqu'à** (n=0)

retourner (occ)

**Fin**

**Procédure** remplirTab (Var T: Tab, var k :entier, var somOcc :entier, n :entier)

**Variable** i : entier

**Début**

somOcc ← 0

k ← 0

**pour** i de 1 à n **faire**

    somOcc ← somOcc + Vérif(i)

**si** Vérif(i) ≠ 0 **alors**

        k ← k+1

        T[k] ← i

**FinSi**

**FinPour**

**Fin**

**Procédure** afficherTab (T: Tab, k :entier)

**Variable** i : entier

**Début**

**pour** i de 1 à k **faire**

    écrire (T[i])

**FinPour**

**Fin**

**Algorithme Principal**

**Constante** max=100

**Types** : Tab : Tableau [1..max] de entier

**Variables** : T : Tab

    n , k, somOcc: entier

**Début**

**Répéter**

    Ecrire("donner un entier positif")

    Lire(n)

**Jusqu'à** (n > 0 et n ≤ max)

    remplirTab (T, k, somOcc, n)

    Ecrire(" Nbre de chiffres 1 : ", somOcc)

    Ecrire(" N ", k)

    afficherTab (T, k)

**Fin**

## EXERCICE 2

**Procédure** saisirT (Var T: Tab)

**Variable** i: entier

**Début**

    Ecrire("donner la première lettre majuscule")

**Répéter**

        Lire(T[1])

**Jusqu'à** (T[1] = 'A')

**Pour** i de 2 à 26 **faire**

        Ecrire("donner la lettre majuscule suivante")

**Répéter**

            Lire(T[i])

**Jusqu'à** (T[i] = Succ (T[i-1]))

**finPour**

**Fin**

Ou bien

**Début**

**Pour** i de 1 à 26 **faire**

    Ecrire("donner une lettre majuscule ")

**Répéter**

        Lire(T[i])

**Jusqu'à** (ord(T[i]) = (64+i))

**finPour**

**Jusqu'à** (T[i] = chr(64+i))

**Fin**

**Procédure** saisirT1 (Var T: Tab)

**Variable**

    i, n1: entier

    c : caractère

    trouve : booléen

**DEBUT**

**Répéter**

Ecrire("Taper une Majuscule")

Lire(c)

Ou bien

Ou bien

**Jusqu'à** (EstMajuscule(c))

**Jusqu'à** ( c >= 'A' et c <= 'Z' )

**Jusqu'à** ( c = Majus (c) )

T[1] ← c

n1 ← 1 //taille réelle de T (nb de cases remplies)

**Répéter**

Ecrire("Taper une Majuscule")

Lire(c)

**Si** (EstMajuscule(c)) **alors**

trouve ← faux

i ← 0

**Répéter**

i ← i+1

trouve ← (T[i]=c)

**Jusqu'à** (trouve = vrai ou i=n1 )

**si** (non(trouve)) **alors**

n1 ← n1+1

T[n1] ← c

**FinSi**

**FinSi**

**Jusqu'à** (n1=10)

**FIN**

i ← 1

**Répéter**

Trouve ← (T[i]=c)

i ← i+1

**Jusqu'à** (trouve = vrai ou i=n1+1)

Ou bien

//2ème version de la procédure saisirT1 en utilisant la fonction distinct

**Fonction** distinct (T : Tab, c : caractère, i :entier) :booléen

**Variables** : j: entier

d : booléen

**DEBUT**

d ← vrai

j ← 0

**Répéter**

j ← j+1

d ← (T[j] ≠ c)

**Jusqu'à** (d = faux ou j=i )

Retourner (d)

**FIN**

**Procédure** saisirT1 (Var T: Tab) //2ème version

**Variable** i: entier

**DEBUT**

**Répéter**

Ecrire("Taper une Majuscule")

Lire(T[1])

**Jusqu'à** (EstMajuscule(T[1]))

**pour** i de 2 à 10 **faire**

**Répéter**

Ecrire("Taper une Majuscule")

Lire(T[i])

**Jusqu'à** (EstMajuscule(T[i]) et distinct (T, T[i], i-1))

**finpour**

**FIN**

**DEBUT**

j ← 1

**Tantque** ( j <= i et T[j] ≠ c) **faire**

j ← j+1

**FinTQ**

Retourner (j=i+1)

**FIN**

Ou bien

Ou bien

n1 ← 1 //taille réelle de T (nb de cases remplies)

**Répéter**

**Répéter**

Ecrire("Taper une Majuscule")

Lire(c)

**Jusqu'à** (EstMajuscule(c) et distinct (T,c,n1))

n1 ← n1+1

T[n1] ← c

**Jusqu'à** (n1=10)

**Procédure** remplirT2 (Var T2: Tab, T : Tab, T1 :Tab)

**Variable**

i,j, n2: entier

trouve : booléen

**DEBUT**

```

n2 ← 0 // taille réelle de T2 (nb de cases remplies)
pour i de 1 à 26 faire //parcourir T
    trouve ← faux
    j ← 0
    Répéter //parcourir T1
        j ← j+1
        trouve ← (T[i] = T1[j])
    Jusqu'à (trouve = vrai ou j=10 )

    Si (non(trouve)) alors
        n2 ← n2+1
        T2[n2] ← T[i] //mémoriser le reste dans T2
    FinSi
finPour
FIN

```

### Algorithme Principal

#### Types :

Tab : Tableau [1..26] de caractère

//Tab1 : Tableau [1..26] de caractère

//Tab2 : Tableau [1..10] de caractère

//Tab3 : Tableau [1..16] de caractère

#### Variables :

T, T1, T2 : Tab

#### Début

saisirT(T)

saisirT1(T1)

remplirT2(T2, T, T1)

#### Fin

### EXERCICE 3

#### Procédure saisirN (Var n : entier)

##### Début

##### Répéter

Ecrire("donner un entier entre 3 et 20")

Lire(n)

**Jusqu'à** (n ≥ 3 et n ≤ 20)

##### Fin

#### Procédure saisirT (Var T: Tab, n : entier)

##### Variable

i: entier

##### Début

##### Pour i de 1 à n faire

Ecrire("donner un entier")

Lire(T[i])

##### finPour

##### Fin

#### Procédure inverserT (Var T: Tab, n : entier)

##### Variable

i, x: entier

##### Début

##### Pour i de 1 à n div 2 faire

x ← T[i]

T[i] ← T[n-i+1]

T[n-i+1] ← x

##### finPour

##### Fin

n2 ← 0

Ou bien

**pour** i de 1 à 26 **faire** //parcourir T

**Si** (distinct (T1, T[i], 10)) **alors**

n2 ← n2+1

T2[n2] ← T[i] //mémoriser le reste dans T2

**FinSi**

**finPour**

Ou bien

i ← 1, j ← n

**Tantque** (i < j) **faire**

x ← T[i]

T[i] ← T[j]

T[j] ← x

i ← i+1

j ← j-1

**finTQ**

**Fonction** VérifierInclusT (T1: Tab, n1 :entier, T2: Tab, n2 : entier) : booléen

**Variable** i, j: entier  
trouve : booléen

**Début**

//vérifier l'inclusion des tableaux (T1 dans T2)

i ← 0

**Répéter**//parcourir T1

i ← i+1

trouve ← faux

j ← 0

**Répéter**//parcourir T2

j ← j+1

trouve ← (T1[i] = T2[j])

**Jusqu'à** (trouve = vrai ou j = n2 )

**Jusqu'à** (trouve = faux ou i = n1 )

**Retourner** trouve

**Fin**

**Algorithme Principal**

**Type** : Tab : Tableau [1..20] de entier

**Variables**

n1, n2: entier

T1, T2 : Tab

**Début**

saisirN(n1)

saisirT(T1, n1)

saisirN(n2)

saisirT(T2, n2)

inverserT(T1, n1)

inverserT(T2, n2)

**Si** (VérifierInclusT(T1, n1, T2, n2)) **alors**

Ecrire("T1 est inclus dans T2 ")

**Sinon**

Ecrire("T1 n'est pas inclus dans T2 ")

**Finsi**

**Fin**

**EXERCICE 4**

**Procédure** remplirTab (var T: Tab, n :entier)

**Variable** i : entier

**Début**

**pour** i de 1 à n **faire**

**répéter**

lire (T[i])

**jusqu'à** (T[i] > 0)

**FinPour**

**Fin**

**Procédure** elimRedondance (var T: Tab, n :entier)

**Variable** i, j : entier

**Début**

**pour** i de 1 à n-1 **faire**

**Si** (T[i] ≠ 0) **alors**

**pour** j de i+1 à n **faire**

**si** (T[i] = T[j]) **alors**

T[j] ← 0

**Finsi**

**FinPour**

**Finsi**

**FinPour**

**Fin**

Ou bien

i ← 1, j ← 1

**Tantque** (i ≤ n1 et j ≤ n2) **faire**

**Si** (T1[i] ≠ T2[j]) **alors**

j ← j+1

**Sinon**

i ← i+1

j ← 1

**FinSi**

**finTQ**

**Retourner** (i = n1 + 1)

**Procédure** arrangerTab (var T: Tab, n :entier, var k : entier)

**Variable** i, j : entier

**Début**

i ← 1

j ← 1

k ← n // nb d'éléments non nuls

**répéter**

  si (T[i]=0) alors

    k ← k-1

    j ← i

**répéter** //on cherche le 1<sup>er</sup> élément non nul, après la case d'indice i, pour le permuter avec T[i]

      j ← j+1

**jusqu'à** (j=n+1 ou T[j]≠0)

    si (j≠n+1) alors // T[j]≠0 //on permute T[i] et T[j]

      x ← T[i]

      T[i] ← T[j]

      T[j] ← x

**Finsi**

**Finsi**

  i ← i+1

**jusqu'à** (i=n+1 ou j=n+1)

**Fin**

**Procédure** afficherTab (T: Tab, n :entier)

**Variable** i : entier

**Début**

**pour** i de 1 à n **faire**

  écrire (T[i])

**FinPour**

**Fin**

**Algorithme** Principal

**Constante** max=100

**Type** : Tab : Tableau [1..max] de entier

**Variables** : T : Tab

  n , k: entier

**Début**

**Répéter**

  Ecrire("donner un entier positif")

  Lire(n)

**Jusqu'à** (n>=1 et n<=max)

  remplirTab (T, n)

  elimRedondance (T, n)

  arrangerTab (T, n , k )

  afficherTab (T, k)

**Fin**

## EXERCICE 5

**Procédure** saisirT (Var T: Tab, n : entier)

**Variable** i: entier

**Début**

**Pour** i de 1 à n **faire**

  Ecrire("donner un entier positif")

**Répéter**

    Lire (T[i])

**Jusqu'à** (T[i]>=0)

**finPour**

**Fin**

// éliminer 3<sup>ème</sup> occurrence du sous tableau B dans le tableau A

**Procédure** suppOcc3 (Var A: Tab, n1 : entier, B: Tab, n2 : entier, var occ :entier)

**Variable** i,j,k: entier

**Début**

```

occ ← 0 // Compteur du nombre d'occurrences de B trouvées dans A.
i ← 1 // Position actuelle dans A où la recherche commence.
tantque (i ≤ n1 - n2 + 1) faire // possibilité d'avancer n2 cases (taille de B) dans A
    j ← 1 // Position actuelle dans B lors de la comparaison.
    k ← i // Position dans A correspondant à la comparaison avec B[j].
    tantque (j ≤ n2 et A[k] = B[j]) faire
        k ← k + 1
        j ← j + 1
    finTantque

    Si (j = n2 + 1) alors // une occurrence de B est trouvée
        occ ← occ + 1
        i ← i + n2 (/ i ← k) // on passe à la position où on poursuivra la recherche d'une nouvelle occurrence de B
    Sinon
        i ← i + 1 // on passe à la case suivante de A pour rechercher de nouveau une occurrence de B
    finSi

Si (occ = 3) alors // décalage pour supprimer la 3ème occurrence de B
    tantque (i ≤ n1) faire
        A[i - n2] ← A[i]
        i ← i + 1
    finTantque
finSi
finTantque
Fin

```

**Algorithme Principal****Constante** max = 50**Type** : Tab : Tableau [1..max] de entier**Variables**

n1, n2, i, occ : entier

A, B : Tab

**Début**

Ecrire(" donner le nombre des éléments pour le tableau A ")

**répéter**

Lire(n1)

**Jusqu'à** (n1 > 0 et n1 ≤ max)

saisirT(A, n1)

Ecrire(" donner le nombre des éléments pour le tableau B ")

**répéter**

Lire(n2)

**Jusqu'à** (n2 > 0 et n2 ≤ max)

saisirT(B, n2)

suppOcc3 (A, n1, B, n2, occ)

//affichage du tableau A

**Si** (occ = 3) **alors**pour i de 1 à n1 - n2 **faire** // la troisième occurrence du tableau B est supprimée

Ecrire (A[i])

**finPour****sinon**pour i de 1 à n1 **faire** // pas de suppression effectuée

Ecrire (A[i])

**finPour****Finsi****Fin**

## EXERCICE 6

Mat : Tableau [1..100, 1..100] de entier

**Fonction** matDiagonale (M: Mat, n : entier) : booléen

**Variable** i,j: entier

diag : booléen

**Début**

diag  $\leftarrow$  vrai

i  $\leftarrow$  1

**Répéter**

j  $\leftarrow$  1

**Répéter**

**Si** (i  $\neq$  j et M[i,j]  $\neq$  0) alors

diag  $\leftarrow$  faux

**finSi**

j  $\leftarrow$  j+1

**Jusqu'à** (diag = faux **ou** j=n+1 )

i  $\leftarrow$  i+1

**Jusqu'à** (diag = faux **ou** i=n+1)

**Retourner** diag

**FIN**

**Début**

**Pour** i de 1 à n **faire**

**Pour** j de 1 à n **faire**

**Si** (i  $\neq$  j et M[i,j]  $\neq$  0) **alors**

Retourner faux

**FinSi**

**finPour**

**finPour**

Retourner vrai

**Fin**

Ou bien